

An ensemble-based predictive mutation testing approach that considers impact of unreached mutants

Alireza Aghamohammadi  and Seyed-Hassan Mirian-Hosseiniabadi^{*,†} 

Department of Computer Engineering, Sharif University of Technology, Tehran, Iran

SUMMARY

Predictive mutation testing (PMT) is a technique to predict whether a mutant is killed, using machine learning approaches. Researchers have proposed various methods for PMT over the years. However, the impact of unreached mutants on PMT is not fully addressed. A mutant is unreached if the statement on which the mutant is generated is not executed by any test cases. We aim at showing that unreached mutants can inflate PMT results. Moreover, we propose an alternative approach to PMT, suggesting a different interpretation for PMT. To this end, we replicated the previous PMT research. We empirically evaluated the suggested approach on 654 Java projects provided by prior literature. Our results indicate that the performance of PMT drastically decreases in terms of area under a receiver operating characteristic curve (AUC) from 0.833 to 0.517. Furthermore, PMT performs worse than random guesses on 27% of the projects. The proposed approach improves the PMT results, achieving the average AUC value of 0.613. As a result, we recommend researchers to remove unreached mutants when reporting the results. © 2021 John Wiley & Sons, Ltd

Received 30 April 2021; Accepted 9 May 2021

KEY WORDS: software testing; mutation testing; predictive mutation testing; machine learning

1. INTRODUCTION

Mutation testing is the approach of generating several faulty versions of a program by means of applying syntactic changes to source code [1-3]. The syntactic changes are systematically produced using transformation rules named *mutation operators*. For instance, replacing `<=` with `<`, `==` with `!=` and `&&` with `||` are among the typical mutation operators. Each faulty version of the program is called a *mutant*. Mutation testing is generally used to assess the effectiveness of a test suite [4-17]. The test suite is executed against generated mutants. A mutant is said to be *killed* if there is at least one test case that fails. A mutant is considered as the *survived* mutant if all test cases pass. The ratio of the number of killed mutants to the total number of non-equivalent mutants is the mutation score of the test suite [18]. Mutation score indicates the quality of a test suite. Mutation testing also helps developers recognize the test suite limitations, showing which mutants have not been detected.

Despite the benefits of mutation testing, it has several problems [19-22]. One problem is in regard to *equivalent* mutants. Whereas equivalent mutants have different syntax, the outputs are the same as the original program. Unfortunately, determining equivalent mutants in a theoretical way is undecidable [23-25]. Another problem is concerned with *redundant mutants*. A mutant M_1 is redundant if killing another mutant M_2 results in killing M_1 . Papadakis *et al.* [26] showed that about 5% of generated mutants are non-redundant. They suggested that redundant mutants should be removed. Another issue with mutation testing is in regard to an overwhelming number of generated

*Correspondence to: Seyed-Hassan Mirian-Hosseiniabadi, Department of Computer Engineering, Sharif University of Technology, Tehran, Iran.

†E-mail: hmirian@sharif.edu

mutants; thus, mutation testing is computationally expensive [27,28]. As an example, it costs hours to conduct mutation testing even for a medium-sized project like Commons Lang, which has fewer than 39,000 mutants [29].

Researchers have proposed various techniques to reduce the cost of mutation testing such as decreasing the number of executed test cases [30–32], limiting the creation of specific mutants [33–36] and selecting certain types of mutants [37,38]. Predictive mutation testing (PMT) [29,39,40] is a method to predict whether a mutant is killed by means of using machine learning techniques without conducting the mutation testing process.

Zhang *et al.* [29,40] are the first ones who introduce the PMT idea. They extracted software metrics and used them as input to machine learning algorithms. The machine learning models then are exploited to predict the mutant execution results. The machine learning models learn some patterns from existing data that, for example, there seems to be a high probability a mutant is killed when executed more than 100 times. Therefore, for a new mutant with more than 100 executions, it predicts the killing of the mutant.

Mao *et al.* [39] extended the Zhang *et al.* work by increasing the number of projects, from 163 to 654, increasing the number of software metrics, from 14 to 95, and performing PMT under the *cross-project* setting, which proved to be a harder problem than the cross-version setting. The cross-project PMT exploits a machine learning model to predict mutant execution results for totally unseen new projects. We discuss the details of PMT in Section 2.

Although the results of the Zhang *et al.* work and the Mao *et al.* work seem promising, they [29,39] both have an assumption that may impact the results. This assumption is *unreached mutants*. A mutant is unreached if the statement on which the mutant is generated is not executed by any test cases [41,42]. As unreached mutants certainly survive, adding unreached mutants to the final results of PMT might represent the results better than in reality.

To better understand the impact of unreached mutants, consider the motivating example illustrated in Figure 1. The method `numberOfPositiveElements` counts the number of positive elements in an integer array list (i.e. `numbers`). We write two test cases for the method. In the first

```

1. public int numberOfPositiveElements(int[] numbers) {
2.     int count = 0;
3.     for (int number: numbers) {
4.         if (number > 0) { // "number >= 0" "number < 0"
5.             count++; // "count--"
6.         }
7.     }
8.     return count; // "(count == 0 ? 1 : 0)"
9. }

@Test
public void onlyOneNumber() {
    int[] numbers = {0};
    assertEquals(0, numberOfPositiveElements(numbers));
}

@Test
public void allAreNegative() {
    int[] numbers = {-1, -5};
    assertEquals(0, numberOfPositiveElements(numbers));
}

```

Figure 1. Motivating example.

one, the variable `numbers` equals `{0}`. In the second one, it equals `{-1, -5}`. Assume `numberOfPositiveElements` has four mutants: `~number>=0~`, `~number<0~`, `~count--~` and `~(count==0?1:0)~`. We would like to predict the mutation testing results before conducting the whole process. Both two test cases execute the mutant `~(count==0?1:0)~`. Moreover, `~number>=0~` is executed by both test cases. Interestingly, although executed once by the first test case, `~number<0~` is not killed by that test case. On the other hand, `~number>=0~` and `~number<0~` are executed two times by the second test case and are classified as killed. This gives us an intuition that a mutant executed more times than another has more chances of being killed. As neither test case executes the statement `~count++~` (in the original program), the mutant `~count--~`, which is generated on that statement, is not executed. Therefore, it is trivial that the mutant `~count--~` is classified as survived. We argue that there is no point in including this mutant (or in general unreachable mutants) in the prediction process. Moreover, the well-known mutation testing tools like Major and PIT exclude unreachable mutants from mutation score computation. Adding the `~count--~` prediction to the final results could artificially increase the PMT performance. Suppose PMT correctly predicts one of the reached mutants (e.g. the mutant `~(count==0?1:0)~`). If we do not remove the unreachable mutant from the prediction set and PMT correctly predicts that mutant, then the PMT performance is $50\% \left(\frac{1+1}{4} \right)$. However, the true performance of PMT is not 50%. Instead, it is $33\% \left(\frac{1}{3} \right)$.

The main purpose of our research is to examine the impact of unreachable mutants. Specifically, we would like to see the performance of PMT on reached mutants. For this purpose, we replicated the Mao *et al.* study [39]. The results show that although PMT results seem promising, they performed poorly on reached mutants. In 27% of the unseen projects, the Mao *et al.* work performed worse than random guesses. Given that unreachable mutants affect the results, one reasonable question may arise as to whether there is an approach that performs better than random guesses in every test project. The answer to this question is affirmative. We propose the approach named ensemble predictive mutation testing (EPMT) exploiting random forest and gradient boosting to improve PMT results. We empirically evaluated EPMT in terms of unbiased metrics towards the imbalanced data set (we define those evaluation metrics in Section 3). Finally, we would like to capture the most important features in PMT. In particular, we would seek any difference in the interpretation when considering the impact of unreachable mutants. The results show that the feature ranks in PMT change significantly, suggesting that the prior research interpretations are misleading. Furthermore, we provide an explanation for the suggested machine learning model so as to describe why a mutant is predicted to be killed or alive.

In summary, our contributions are as follows:

- We investigate the impact of unreachable mutants on the results of PMT, showing that the performance of PMT decreases significantly from 0.83 to 0.51 in terms of the area under a receiver operating characteristic (AUC) curve.
- We propose an approach based on random forest and gradient boosting, considering the effect of unreachable mutants and improving the PMT results in terms of various unbiased evaluation metrics.
- We examine the most important features in PMT and find some contradictions when considering the impact of unreachable mutants.
- We provide an interpretable explanation for EPMT, which can be served as guidance for developers to understand why a mutant is killed or alive.
- Similar to the Mao *et al.* work and the Zhang *et al.* work [29,39], we make all our source code publicly available for other researchers.

The rest of the paper is organized as follows. In Section 2, we explain background information to understand PMT. Section 3 describes our research methodology. In Section 4, we express the results of our research questions. Section 5 reviews threats to the validity of this paper. We review related work in Section 6. We conclude and present our future plans in Section 7.

2. BACKGROUND: PREDICTIVE MUTATION TESTING

In this section, we provide the necessary information to understand this paper. Specifically, we present PMT and the concepts we use throughout the paper. We review the Zhang *et al.* work and the Mao *et al.* work because this paper is heavily based on those [29,39,40].

Zhang *et al.* [29] introduced the notion of PMT. PMT is the approach based on machine learning techniques that predicts the execution results of mutants, namely, killed or alive, without conducting mutation testing. The idea is anticipating the mutant execution results beforehand to reduce the overhead of mutation testing. They collected 163 real-world Java projects and demonstrated that PMT obtained promising results under the *cross-version* and *cross-project* settings [29]. The cross-version setting aims at predicting the mutant execution results using the earlier versions of a program. The cross-project setting concerns the prediction of mutant execution results using a different set of projects.

They employed various supervised learning such as naïve Bayes [43], support vector machine (SVM) [44], decision tree (J48) [45] and random forest. The results provided in the Zhang *et al.* paper show that random forest outperformed other machine learning models (i.e. naïve Bayes, SVM and J48) in PMT [29]. They defined 14 different features for each project as input to the machine learning model. These features can be categorized as either static or dynamic. The dynamic features such as `numExecuted` and `numTestCover` are related to the run-time behaviour of the program. The first one determines the number of times each mutant is executed. The latter indicates how many test cases execute the mutant. As well as two previous dynamic features, they presented two other dynamic ones, namely, `numAssertInTM` and `numAssertInTC`. The first one refers to the number of assertions in test methods that exercise the mutant. The latter indicates the number of assertions in test classes that exercise the mutant. The static features such as `infoComplexity` and `LOC` are regarding the compile-time information of source code. Interested readers can refer to the original paper to see the full list of feature names accompanied by definition [29].

Zhang *et al.* used Major and PIT, two well-known mutation testing tools to generate mutants. Researchers have studied both tools extensively [46-54]. Major and PIT are also used in industry. PIT manipulates Java bytecode to generate mutants. On the other hand, Major changes the abstract syntax tree to produce mutated versions. Zhang *et al.* [29] used the whole mutation operators of PIT and Major. Generated mutants are created by *method-level* operators, which are applied to a method body [55].

Because each project has a different proportion of killed mutants, Zhang *et al.* also considered three strategies to deal with the imbalanced nature of the projects, namely, *undersampling*, *cost-sensitive* and *naïve* [29]. The undersampling approach removes the dominant category in each project to balance the number of killed and survived mutants. The cost-sensitive method assigns more weight to the minority during the learning process. The naïve strategy does not perform rebalancing techniques or cost-sensitive methods. They concluded that undersampling and cost-sensitive strategies do not help in improving the effectiveness of PMT because their results failed to improve; thus, the naïve approach is the best strategy to deal with an imbalanced data set.

Then Mao *et al.* [39] extended PMT, introducing 654 real-world Java projects and 95 features. Their data set, machine learning classifications and features cover the Zhang *et al.* work [29]. The Mao *et al.* idea was that enriching the features extracted from source code would improve PMT effectiveness. This is due to the fact that software metrics play an important role in research areas such as defect prediction [56-61], mutation testing [14,27,62,63] and testability [64].

They used the same four dynamic features provided in the Zhang *et al.* work. The remaining 91 features are the static ones. They gathered two categorical features, namely, `MutatorClass` and `returnType`. `MutatorClass` shows the type of mutation operator. `returnType` refers to the return type of the method in which the mutant is located. The static features are computed by the JHawk tool [65]. Interested readers could study the original paper to see the full name of features as well as the definition of each one [39]. However, we introduce the top 30 features that are important in this paper. Table I lists the 30 features as well as the definition of each one. The `pp`, `cc` and `mm` prefixes refer to package, class and method-level metrics.

Table I. The list of features accompanied by their definitions.

Name	Definition
numExecuted	The number of times each mutant is executed
MutatorClass	The type of mutation operator
numAssertInTC	The number of assertions in test classes that execute the mutant
numAssertInTM	The number of assertions in test methods that execute the mutant
numTestCover	The number of distinct test cases that cover the mutant
ppavcc	Package-level cyclomatic complexity
cchalsteadCumulativeBugs	Class-level cumulative Halstead bugs
ppRVF	The total number of variables referred in the package
ppnumberOfMethods	The number of methods each package has
ppnumberOfClasses	The number of classes each package has
ppmaintainabilityIndexNC	The package-level maintainability index without considering comments
ppfanout	The package-level fan out
ccmaintainabilityIndex	The class-level maintainability index
mmhalsteadDifficulty	The method-level Halstead difficulty
ppabstractness	The package-level abstractness
ppmaintainabilityIndex	The package-level maintainability index
ccexternalMethodCalls	The total number of method invocations in the class
mminstanceVariablesReferenced	The number of fields referred in the method
ccimportedPackages	The number of packages imported in the class
ppdistance	The package-level distance metric
returnType	The type of the return value of the method
ccfanIn	The class-level fan in
ppfanin	The package-level fan in
pploc	The package-level lines of code
ccmaintainabilityIndexNC	The class-level maintainability index without considering comments
mmexternalMethodsCalled	The number of method invocations in the method
ppinstability	The package-level instability
ppmaxcc	Maximum cyclomatic complexity value in the package
mmvariablesReferenced	The number of variables referred in the method
ccunweightedClassSize	The sum of the number of methods and fields

The definition of static features is borrowed from the JHawk tool [64].

Mao *et al.* trained 11 classification algorithms to assess PMT under the cross-project setting. Random forest, extremely randomized trees [66], decision stumps [67,68], decision trees (C4.5) [69], deep forest [70], SVM [44], K-nearest neighbours [71], naïve Bayes [43], MLP [72], deep MLP [72] and convolutional neural networks (CNN) [73] are the supervised learning algorithms they used in the paper. Similar to the Zhang *et al.* work [29], random forest also outperformed the state-of-the-art techniques in the extended version of PMT.

However, there is an assumption in both the Zhang *et al.* work and the Mao *et al.* work [29,39], which may affect the results. Every unreached mutant certainly survives; thus, there is no need to include these mutants in the prediction process. Previous research has not fully considered the effect of those mutants; nonetheless, it might reduce the PMT effectiveness and inflate the results.

To better understand this, suppose we have 100 mutants in the test set; 60 of them are unreached ($\text{numExecuted} = 0$) and the remaining are reached ($\text{numExecuted} > 0$). Assume PMT correctly predicts 20 reached mutants (20 mutants from the 40 ones). PMT also may predict correctly all the unreached mutants (it just needs to check whether numExecuted is zero or not). If we do not remove the unreached mutants ($\text{numExecuted} = 0$) from the test set, then the model performance is $80\% \left(\frac{20 + 60}{100} \right)$. However, the true performance of PMT is not 80%. Instead, it is $50\% \left(\frac{20}{40} \right)$. Our motivation in this paper is to explore the influence of unreached mutants on the final results (i.e. on the performance of PMT on reached mutants) and to further improve PMT results while considering unreached mutants. In this paper, we focus on the cross-project

setting because the results of the Mao *et al.* paper and the Zhang *et al.* work indicate that the cross-project setting is more difficult than the cross-version one [29,39]. It may be due to the fact that in a cross-version setting, we have access to the history of mutant results. However, in a cross-project setting, we do not have such information at our disposal.

3. METHODOLOGY

In this section, we explain our research methodology. We follow the goal-question-metric paradigm to structure the research methodology [74]. The main purpose of this study is to unveil the impact of unreached mutants on PMT. First, we define three research questions as well as the motivation behind them. Then we explain the evaluation metrics used to answer the research questions. Having explained the characteristics of subject programs, we finally provide our experimental design for answering each individual question.

3.1. Research questions

We pose three research questions to realize the impact of unreached mutants, to assess the quality of EPMT and to interpret important factors in PMT.

- RQ1: To what extent the unreached mutants affect the results of PMT? RQ1 tries to investigate the effects of unreached mutants on the previous research results. We hypothesize that this assumption could drop significantly the performance of the machine learning models on reached mutants. If this is the case, then we propose EPMT that shows that there is a machine learning model capable of performing better than random guesses on every unseen project. The next research question assesses the quality of EPMT.
- RQ2: How well does EPMT perform compared with state-of-the-art approaches? RQ2 assesses EPMT in comparison with other PMT methods, namely, the Mao *et al.* work and the Zhang *et al.* work [29,39]. We empirically show that EPMT not only outperforms the previous PMT approaches but also EPMT is better than random guesses in every test project.
- RQ3: Which features are important in the process of predicting the execution results of mutants? In RQ3, we would like to capture the most important features in PMT. In particular, we would seek any difference in the interpretation of PMT before and after considering unreached mutants. If we could show that the ranks of features in predicting such mutants change significantly, it signifies that the prior research interpretations are misleading.

3.2. Evaluation metrics

Here, we define our metrics to answer the research questions. We use three evaluation metrics, all of which are unbiased towards an imbalanced data set. The metrics for imbalanced data are important because other metrics such as precision, recall and F_1 could be misleading in that condition [75]. Before explaining those metrics, we need to define some terminologies. True positive (TP) refers to the number of times when PMT correctly predicted that a mutant would be killed, true negative (TN) is the number of times when PMT correctly predicted that a mutant would be alive, false positive (FP) refers to the number of times when PMT mistakenly predicted that a mutant would be killed and false negative (FN) is the number of times when PMT incorrectly predicted that a mutant would be alive.

AUC computes the area under an ROC (receiver operating characteristic) curve [76]. The ROC curve illustrates the relationship between the true positive rate $\left(\text{i.e. } \frac{TP}{TP + FN}\right)$ and the false positive rate $\left(\text{i.e. } \frac{FP}{FP + TN}\right)$ while changing the discrimination threshold. AUC is an evaluation metric ranging in $[0, 1]$. The AUC value of one signifies the perfect predictive model. The AUC value of 0.5 is similar to random guesses. The AUC value of less than 0.5 is worse than random guesses.

MCC stands for Matthews correlation coefficient. Unlike F_1 , MCC takes true negative into its account [77]. Equation (1) determines how MCC is computed. MCC is located in the range of $[-1, 1]$. The MCC value of one means the perfect predictive model, while the MCC value of zero means random guesses. Note that the negative MCC value signifies that the model performs worse than random guesses.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (1)$$

BA stands for balanced accuracy. BA is a variation of accuracy that solves the problem of traditional accuracy under the imbalanced settings [78]. In this paper, we used its adjusted version rescaled to the range of $[-1, 1]$. Adjusted BA is calculated according to Equation (2). Similar to MCC, the same interpretation holds for this evaluation metric.

$$BA = \frac{TP}{TP + FN} + \frac{TN}{TN + FP} - 1 \quad (2)$$

3.3. Subject programs

In this paper, we used the same 654 Java projects presented by Mao *et al.* [39]. Mao *et al.* gathered this data set in a systematic way. First, they mined over 8000 Java projects, each of which has at least 20 forks in Github. Then they removed projects that do not support Maven as a build tool. About 5000 projects fulfilled this requirement. They also removed projects that do not use JUnit or TestNG. Fewer than 3000 projects remained. Finally, they kept those projects on which mutation testing tools can be run without difficulty and reached the 654 final projects. They ensured that their data set covers the Zhang *et al.* subject projects.

The 654 projects come from different domains such as OpenNLP (a natural language processing tool), Commons Lang (utility libraries), Joda Time (time utilities), Fess (an enterprise search server) and Mixer2 (an engine for writing a template). They also vary in size. For example, Mixer2 has more than 450,000 source lines of code (SLOC), 10,000 test cases, 5000 classes, 30,000 methods and 350,000 mutants. On average, the 654 projects have more than 8000 SLOC, 100 test cases, 100 classes, 700 methods and 6000 mutants. The detailed statistics of these projects are available at the Mao *et al.* paper [39].

Figure 2 depicts the histogram of killed mutants for the 654 projects. The x -axis ranges from zero to one, illustrating the mutation score (i.e. the ratio of killed mutants). The y -axis demonstrates the

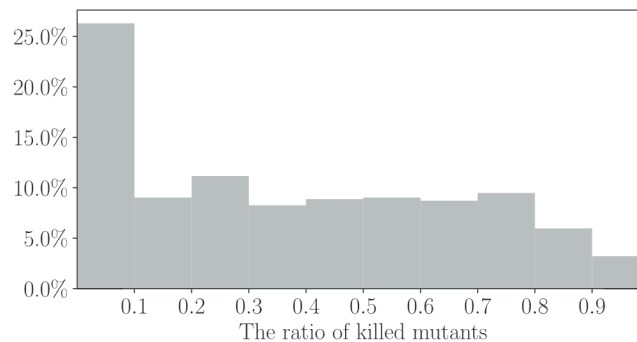


Figure 2. The distribution of killed mutants for the whole data set.

percentage of projects located in each range. For example, about 26% of the projects have the mutation score of less than 0.1. We checked the data and found that about 16% of the projects have line coverage of less than 0.1, and about 12.5% of the projects have line coverage between 0.1 and 0.2. This may explain the reason why 26% of the projects have such low mutation score. In addition, about 3% of the projects have the mutation score of more than 0.9. The distribution indicates that the data set is imbalanced (i.e. the number of killed and survived mutants is not equal per project).

3.4. Experimental design

Here, we explain how we conducted the experiments to answer the research questions.

3.4.1. RQ1: the impact of unreached mutants. In this section, we examine the effectiveness of previous research with respect to PMT by considering the impact of unreached mutants. Every unreached mutant survives. Consequently, including these mutants in the prediction process could inflate the results. Table II shows the number of reached and unreached mutants. Based on this table, about 62% of the total mutants are not executed, which is significantly more than half of all the mutants available in the data set. The quantity of unreached mutants may impact the results of previous research.

To better understand the volume of unreached mutants, we investigate the distribution of unreached mutants in the data set. Figure 3 demonstrates the histogram of unreached mutants. The x-axis ranges from zero to one, showing the percentage of unreached mutants with the bin width of 0.1. The y-axis indicates the percentage of projects lying on each range. For example, projects whose unreached mutants comprise more than 90% of their generated mutants are about 16% of the total projects. In addition, about 55% of the projects are those in which the majority of mutants are unreached. This distribution suggests that we have faced projects with a different rate of unreached mutants.

To investigate the impact of unreached mutants, we replicated the same procedure described by the Mao *et al.* work [39]. We used the same data set and their best machine learning model (i.e. random forest) to explore the effectiveness of PMT. Specifically, we randomly chose 10% of the projects (i.e. 66 projects) out of 654 projects as the test data and investigated the performance of PMT in predicting reached mutants to unveil any inflation that unreached mutants might produce. One resealable question some may ask is what happens if we train the Mao *et al.* model only on reached mutants. If training on reached mutants leads to a better model, then unreached mutants are not even helpful in the learning process. To this end, we also trained the Mao *et al.* model on reached mutants and examined how it would perform.

3.4.2. RQ2: proposed approach. Here, we elaborate upon EPMT. The goal of EPMT is to demonstrate that there is an approach performing better than random guesses in every unseen project. EPMT is based on the idea that combining random forest and gradient boosting, using a rebalancing technique, and eliminating unnecessary features would be of benefit to PMT. Figure 4 illustrates the high-level overview of EPMT. In the following, we detail each step of EPMT.

3.4.2.1. Step 1: collect data

As suggested by other researchers and practitioners [79-81], we split the data into three independent sets, namely, the train (80%), validation (10%) and test (10%) sets. In particular, we randomly chose 522 Java projects for the train set, 66 Java projects for the validation set and 66 Java projects for the

Table II. The number of reached and unreached mutants in the data set.

	Is reached?	The number of mutants	Percentage
	□	1,137,336	37.5
	×	1,894,940	62.5
Total	—	3,032,276	—

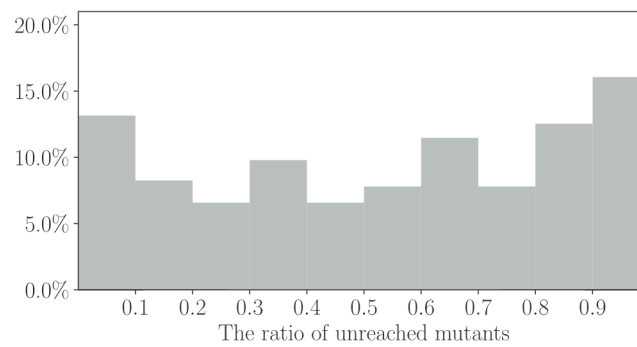


Figure 3. The distribution of unreachable mutants for the whole data set.

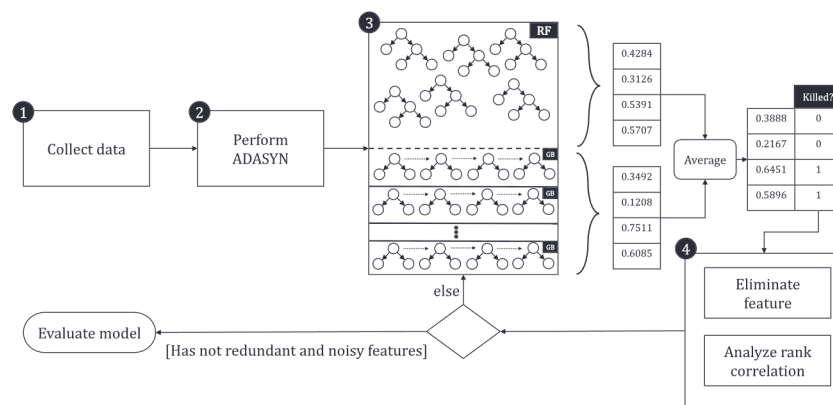


Figure 4. The high-level overview of EPMT.

test set. Note that the 66 projects we chose for the test set are the same as those 66 randomly chosen projects in RQ1. To avoid inflating results, we removed unreachable mutants and considered only those mutants to be executed at least once. The validation set was used to tune model hyperparameters. Note that the test set was not used for tuning hyperparameters. Indeed, the test set did not participate in the hyperparameter tuning process due to avoid bias in the evaluation part. Having found all hyperparameters, we utilized the test set to evaluate the machine learning model.

Table III presents some statistical information with respect to the train, validation and test sets. The number of mutants in the test set is about 5% of the number of mutants in the train set. The ratios of killed mutants are about 67%, 75% and 64% for the train, validation and test sets, respectively. Moreover, Figure 5 illustrates the distribution of unreachable mutants for the train, validation and test sets. The x -axes show the proportion of unreachable mutants per project. The y -axes demonstrate the densities belong to each range. According to this figure, the data set is diverse enough whose projects contain different proportions of unreachable mutants. For example, projects (in the test set) whose unreachable mutants comprise less than 10% of their generated mutants are about 15% of the total test projects.

3.4.2.2. Step 2: perform ADASYN

One of the challenges in today's machine learning problems is to tackle imbalanced data sets [59,82-85]. A data set is said to be imbalanced if one of the dependent variables dominates the sample space. Rebalancing techniques have been shown that improve the performance of predictive models in terms of AUC and recall [59].

In this paper, we used a famous approach named adaptive synthetic sampling approach (ADASYN) to handle the imbalanced data [86]. ADASYN oversamples minority class synthetically while considering the difficulty of those minor instances. It is worth to note that we just rebalanced the train data and did not modify the validation and test sets. The validation and test sets

Table III. Statistical information about the train, validation and test sets.

	Train	Validation	Test
# Mutants	1,011,364	67,496	58,476
# Killed	680,474	50,839	37,843
# Survived	330,890	16,657	20,633

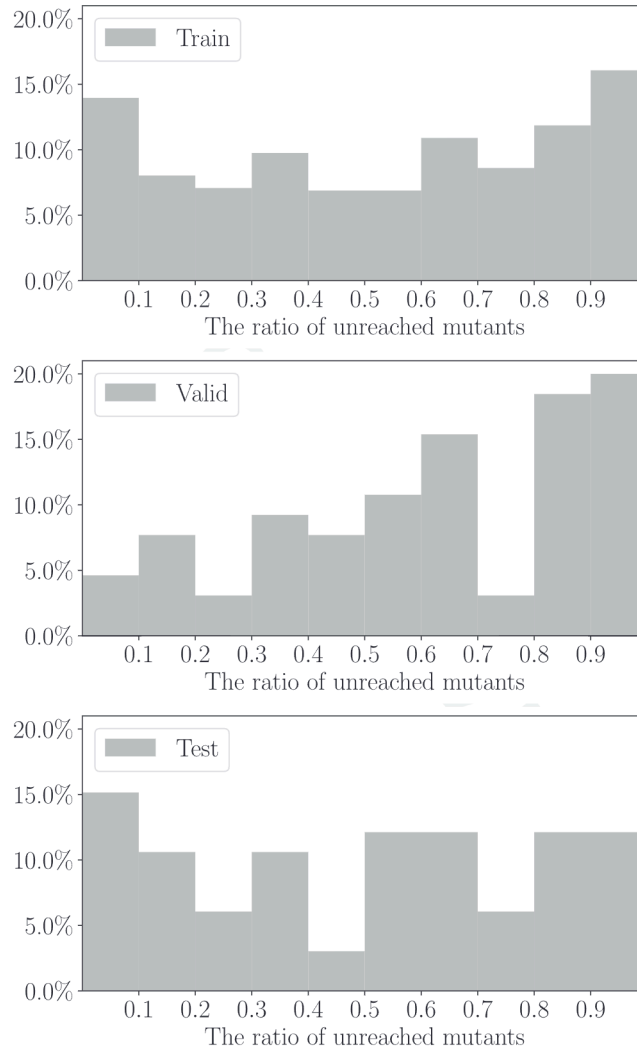


Figure 5. The distributions of unreached mutants in the train, validation and test sets.

are aimed at representing unseen data. By doing so, we ensured that each project in the train set has the same number of killed and survived mutants.

3.4.2.3. Step 3: train the model

Our machine learning model has two folds. The first fold is random forest. Random forest is a *bagging* approach, which tries to train different decision trees on data [87]. Each tree is trained on a different portion of the data. Random forest aggregates the results. In Equation (3), RF is determined based on the average of N different DT_i scores— N is the number of decision trees. One advantage of random forest is reducing the variance of a predictor (i.e. decision tree) [88].

$$\text{RF}(X) = \sum_{i=1}^N \frac{1}{N} \text{DT}_i(X) \quad (3)$$

The second fold is gradient boosting. Boosting is the approach based on the combination of *weak learners* [68]. A weak learner is an estimator that is slightly better than a random guess. For example, decision tree with the depth of one, decision stump, is a weak learner. Boosting makes a decision based on aggregating weak learners. In Equation (4), GB is calculated based on the decision of T weak learners WL — α_i is the weight attributed to WL_i . One limitation of gradient boosting is overfitting [89]. To avoid overfitting, we used bagging. In other words, we created multiple gradient boostings and trained them on different subsets of the data. We averaged the results to reduce the variance.

$$\text{GB}(X) = \sum_{i=1}^T \alpha_i \text{WL}_i(X) \quad (4)$$

Finally, for each data point (each mutant), we obtained two predictions (the probabilities between zero and one). The first one is the estimation of random forest. The second one is the prediction of bagging of different gradient boostings. The final prediction for each mutant is the mean of those two values. If the prediction is greater than 0.5, we predict the mutant is killed. If the prediction is less than 0.5, we predict the mutant survives.

3.4.2.4. Step 4: eliminate features and analyse rank correlation

Noisy features could decrease the performance of the model. We consider a feature as noisy if it has less than 1% *importance* with respect to other features in predicting dependent variables. To obtain the importance of each feature, we used *permutation importance* [90,91]. The permutation importance is an iterative technique in which one of the independent variable values is shuffled in each iteration—with respect to the original data—decorrelating it with the dependent variable. If the performance of the model is improved, it signifies that the feature is noisy, and we remove it. Otherwise, we record the amount of reduction. These numbers show the importance of the features. It is worth mentioning that we assessed the performance of the model based on AUC. In particular, we used the *recursive feature elimination* approach [92]. We had removed the noisy feature and carried out the permutation importance process recursively until all of the features had the importance of more than 1% with respect to other features.

Redundant features could also reduce the performance of the model. Our machine learning model is a tree-based approach. In the tree-based model, the absolute values of one feature are not important, but the relative values are crucial. We exploited Spearman's Rho rank correlation [93]. Spearman's Rho is a non-parametric rank correlation metric, which measures the rank correlation between two variables. We consider a feature F_1 as redundant if there is another feature F_2 in the data having the rank correlation of more than 0.9 with F_1 . In this case, we removed the feature with less permutation importance. In particular, we removed the least important feature that has more than 0.9 Spearman's rank correlation with another feature. Then we tuned the hyperparameters of the model on the validation set. We had carried out Step 3 and Step 4 recursively until all of the highly correlated and noisy features were eliminated. After removing noisy and redundant features, we obtained 30 features. Figure 6 presents the remaining 30 features along with Spearman's Rho rank correlations.

3.4.2.5. Implementation details

We used the `Pandas` library for reading and writing data [94]. Our implementation of the machine learning model is based on `Scikit-Learn`, a powerful Python machine learning framework [95]. There are two categorical features in the independent variables, namely, `MutatorClass` and

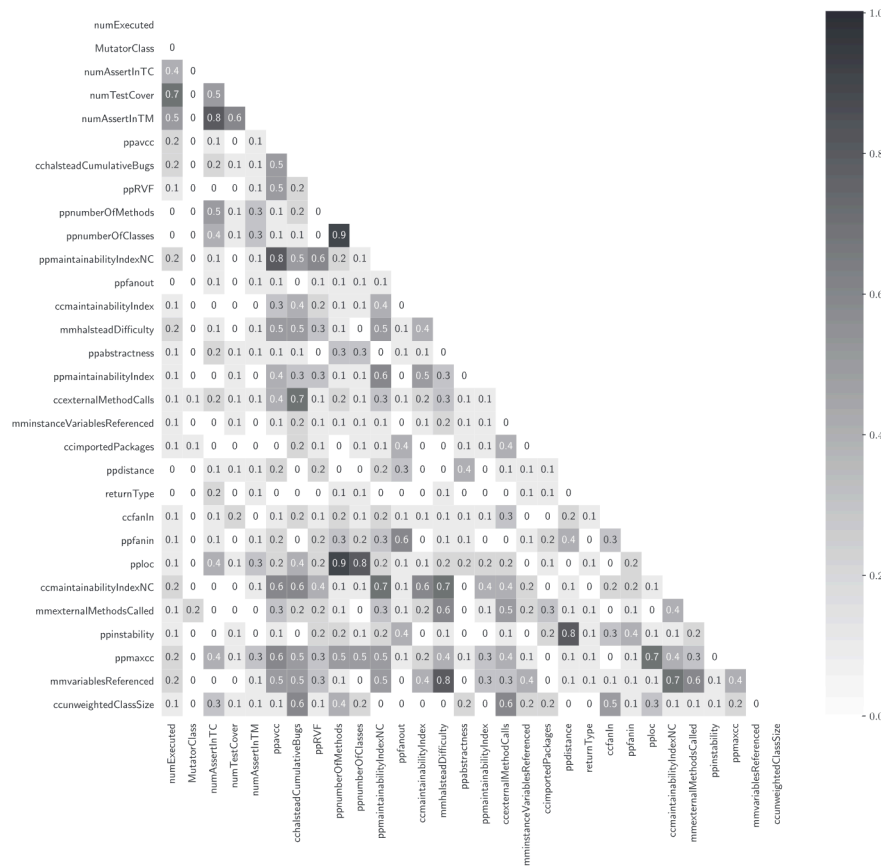


Figure 6. The remaining 30 features accompanied by the Spearman's Rho rank correlations.

`returnType`. `MutatorClass` represents the mutation operator. `returnType` shows the type of the return value of the method. Because EPMT is tree based, it cannot handle categorical features. We transformed those two categorical features into numerical features by exploiting *frequency encoding*. Frequency encoding is an approach that counts the number of values for each category. Then it maps each category to its frequency. We set the number of estimators of random forest to 100 ($n_estimators=100$). Each estimator in each split considers $0.7 \times 30 = 21$ independent variables ($max_features=0.7$). We employed `HistGradientBoostingClassifier` to implement the gradient boosting part. We fitted 50 different gradient boostings and averaged their predictions to reduce the variance.

3.4.3. RQ3: interpretation. We interpret our predictive model by means of investigating the most important features. Besides, we compare the salient features in the PMT approach [39] with our features we have obtained in EPMT. To this end, we exploited the method called permutation importance to get the most influential features [90].

Mao *et al.* [39] extracted 12 features out of 95 independent variables that have the most effect on the ability of PMT to predict the mutant execution results. Similarly, we extracted the 12 most important features out of 30 independent variables before which were introduced in EPMT. Then we compared our 12 features to those 12 features of PMT. We would seek any difference between these two sets of features that may impact the interpretation of ours towards PMT.

Recently, researchers have underscored the need for interpretability of machine learning models especially in the field of software engineering [96,97]. Jiarpakdee *et al.* [97] argued that a global interpretation is not sufficient for both researchers and developers to understand how each individual instance is predicted. Therefore, we need a local interpretation not only to explain the behaviour

of the machine learning model but also to provide insights for developers to take meaningful actions.

To this end, we used local interpretable model-agnostic explanations, LIME, to explain why EPMT makes such a decision for each individual mutant (i.e. killed or alive) [98]. LIME is founded on three ideas. First, we need a simple interpretable model to locally approximate a complex model (local). Second, the users are often unable to process how the complex model predicts (interpretable). Third, it is necessary to interpret any model regardless of how complex it is and what kinds of data it receives (model-agnostic).

Consider a mutant m . LIME first generates random samples, based on extracted features, around m . Then it tries to approximate EPMT by a simple linear regression model, which is interpretable. To this end, LIME uses EPMT to predict labels (i.e. killed or alive) for each generated sample. It assigns weight to samples based on their distances to m . Finally, LIME learns the linear regression model on new weighted samples, approximating EPMT locally by the interpretable machine learning model.

4. RESULTS

In this section, we present the results as well as our own interpretation regarding the three research questions.

4.1. RQ1: the impact of unreached mutants

Table IV shows the performance of the Mao *et al.* work in terms of AUC on both the whole data set and the only reached mutants. Based on the table, AUC drops significantly on reached mutants; that is, the median AUC value decreases from 0.833 to 0.517, which is slightly better than random guesses. It is worth mentioning that in 18 out of 66 (i.e. about 27%) projects, the model performs worse than random guesses. On average, the performance of the original PMT drastically decreases from 0.814 to 0.540 in terms of AUC. It is worth noting that only five projects have AUC of more than 0.6.

Table IV. The performance of PMT on the whole test data and reached mutants in terms of AUC, respectively.

Train data	Test data	Average	Median
Reached + Unreached mutants	Reached + Unreached mutants	0.814	0.833
Reached + Unreached mutants	Reached mutants	0.540	0.517
Reached mutants	Reached mutants	0.558	0.521

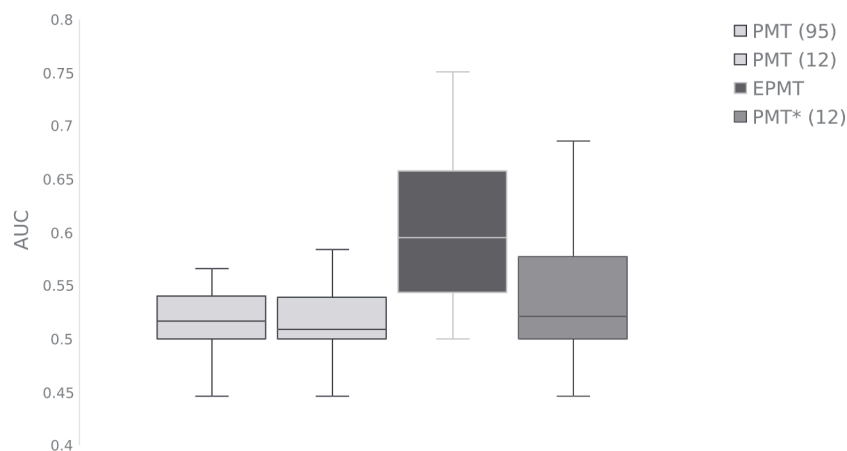


Figure 7. The performance distributions in terms of AUC.

The results provided here suggest that previous PMT performs poorly on reached mutants. To further examine the impact of unreached mutants, we also trained the Mao *et al.* work on reached mutants. The results slightly improve; that is, the median and average AUC values are 0.521 and 0.558, respectively. Note that when the Mao *et al.* work is trained on reached mutants, in 8 out of 66 (i.e. about 12% of the test projects), the model performs worse than random guesses. Also, 11 projects have AUC of more than 0.6. Consequently, unreached mutants not only should be removed from the test set but also should be removed from the train set as they do not help in the learning process. We elaborate upon the performance of PMT as well as the distribution of performance in the following research question results.

4.2. RQ2: proposed approach

Figure 7 depicts the performance distributions of EPMT and original PMT in terms of AUC. PMT (95) means all 95 features were used in the training process. PMT(12) means the top 12 important features were exploited in the training process as suggested by Mao *et al.* [39]. Furthermore, PMT*(12) is the same as the Mao *et al.* work when trained on only reached mutants. EPMT achieved the AUC value of 0.613 on average. Also, the median AUC value of our model is 0.609. It indicates about 13% and 18% improvement regarding the average and median of PMT performance. Unlike PMT(95), PMT(12) and PMT*(12), EPMT gains the AUC value of more than 0.5 in all of the test projects, meaning no project performs worse than random guesses. It is another improvement compared with PMT.

To better understand the difference between EPMT and PMT, we performed multiple statistical tests. We followed the guideline proposed by Demsar [99] to conduct statistical tests for comparing multiple classifiers. Herbold [100] also provided a Python package named Autorank, which implemented the guideline. We used the Shapiro–Wilk test to investigate whether the sample data are normal or not [101]. As there are multiple tests on the same data, we exploited Bonferroni correction to reduce the chance of afflicting the results due to the Type I error. In this case, we used the significance level of $\alpha = \frac{0.05}{3}$ because there are three approaches. The normality tests show that none of them are normal. Therefore, we used the non-parametric Friedman test [102]. We exploited the post hoc Nemenyi test [103] for assessing significant differences between approaches. That is, if the Friedman test rejects the null hypothesis, the post hoc Nemenyi test investigates which differences between the mean ranks of sample data are significant. These are evaluated based on the critical distance (CD); that is, the difference is significant if the difference between mean ranks of sample data is greater than CD (in our case, CD is 0.577). We employed Cliff's Delta for the non-parametric effect size [104]. We interpreted the Cliff's Delta effect size as follows: if $|\delta| < 0.147$, the difference is negligible; if $0.147 \leq |\delta| < 0.33$, the difference is small; if $0.33 \leq |\delta| < 0.474$, the difference is medium; and if $0.474 \leq |\delta|$, the difference is large.

We report the mean rank (MR), median (MD), median absolute deviation (MAD) and confidence interval of the median value (CI) for EPMT, PM(12) and PM(95), respectively. We also report Cliff's Delta (δ) in comparison with the best approach. Table V shows the results of statistical tests.

The Friedman test rejects the null hypothesis that there is no difference between the median values of EPMT, PMT(95) and PMT(12). Therefore, we accept the alternative hypothesis that there is a statistically significant difference between those medians. The post hoc Nemenyi test result indicates that there is no significant difference between PMT(95) and PMT(12). However, the

Table V. The results of statistical tests.

	MR	MD	MAD	CI	δ	Magnitude
EPMT	1.303	0.609	0.079	[0.546, 0.655]	0.000	Negligible
PMT(95)	2.273	0.517	0.025	[0.500, 0.535]	0.636	Large
PMT(12)	2.424	0.509	0.019	[0.500, 0.540]	0.631	Large

The bold highlight the differences between EPMT (the proposed approach) and PMT(95) and PMT(12) (the baselines).

Table VI. The performance of EPMT compared with PMT and each part of our solution.

	BA	MCC	AUC
PMT(95)	0.122	0.138	0.540
PMT(12)	0.113	0.126	0.539
RF(30)	0.131	0.161	0.571
ADASYN + RF(30)	0.169	0.170	0.580
ADASYN + GBs(30)	0.205	0.209	0.605
EPMT	0.230	0.239	0.613

The bold highlight the differences between EPMT (the proposed approach) and PMT(95) and PMT(12) (the baselines).

differences between EPMT, PMT(95) and PMT(12) are significant. The Cliff's Delta values demonstrate that the magnitude of differences between EPMT, PMT(95) and PMT(12) is large.

Table VI shows the results of the evaluation metrics. We calculated the means of these metric values for both PMT and EPMT as well as each part of our solution to better understand the participation rate of different ideas. Based on the results, EPMT outperforms PMT(95) and PMT(12) in all of the aforementioned metrics. We improved the BA value by 88%, MCC value by 73% and AUC value by 13%. Random forest with the 30 features, RF(30), obtained in Step 4 of EPMT, even outperforms both PMT(95) and PMT(12). Note that RF(30) was trained on only reached mutants. When we use ADASYN to balance the data set, the results improve. Interestingly, in contrast to the Zhang *et al.* work, the rebalancing technique improves the results. This is likely owing to the fact that we investigate the PMT performance on reached mutants. Based on this table, the combination of ADASYN and 50 gradient boostings outperforms the combination of ADASYN and random forest. Finally, EPMT, which exploits ADASYN, random forest and 50 gradient boostings, gains the best performance in terms of BA, MCC and AUC. These results show that the combination of bagging and boosting can improve the results (in the PMT setting) significantly.

4.3. RQ3: interpretation

Table VII presents the feature importances of PMT(12) and EPMT. The results specify that 7 out of the top 12 important features are common to the two approaches, namely, numAssertInTC (the number of assertions in test classes that execute the mutant), numExecuted (the number of executions), numTestCover (the number of tests covering the mutant), numAssertInTM (the number of assertions in test methods that execute the mutant), MutatorClass (the mutation operator), ppavcc (the package-level average cyclomatic complexity [105]) and ppmaintainabilityIndexNC (the package-level maintainability index).

However, there are some differences between these two sets. First, numTestCover drops in ranking from the 3rd to the 12th (the last one in EPMT). Second, there are only four features having the feature importance rate of more than 5% in PMT. All of the 12 important features in EPMT have

Table VII. The top 12 important features in PMT and EPMT.

Feature name (PMT)	PMT	Feature name (EPMT)	EPMT
numAssertInTC	0.233	numExecuted	0.165
numExecuted	0.222	mmhalsteadDifficulty	0.118
numTestCover	0.191	MutatorClass	0.091
numAssertInTM	0.143	ppavcc	0.084
MutatorClass	0.047	cchalsteadCumulativeBugs	0.082
ppavcc	0.030	numAssertInTC	0.077
ppmaintainabilityIndexNC	0.027	numAssertInTM	0.076
ppmaintainabilityIndex	0.021	ppnumberOfMethods	0.065
ppnumberOfClasses	0.021	ppmaintainabilityIndexNC	0.060
pptcc	0.020	ccmaintainabilityIndex	0.059
ppnumberOfStatements	0.020	ccmaintainabilityIndexNC	0.058
pploc	0.019	numTestCover	0.055

the feature importance rate of more than 5%, which is statistically non-negligible. Third, although the dynamic features are the most important ones in PMT (top four), static features in PMT play a crucial role in EPMT. For instance, `mmhalsteadDifficulty`, which is the method-level Halstead difficulty metric [106], is the second most important feature in EPMT. Fourth, all static features apart from `MutatorClass`, in PMT, are package-level metrics; however, the static features in EPMT come from a different level of granularity such as method, class and package level. Finally, some of the static features are quite simple in PMT such as package lines of code (`ploc`), the number of package statements (`ppnumberOfStatements`) and the number of classes in the package (`ppnumberOfClasses`). However, the static features in EPMT are quite complex ones, which measure the relationship between elements of the code, for example, `mmhalsteadDifficulty` and `cchalsteadCumulativeBugs`.

Assume a developer uses EPMT. The approach suggests that a mutant would survive. The developer wants to know the reason why the model makes such a decision. Figure 8 shows that how EPMT decides to classify a mutant as alive.

While the darker ones suggest a mutant would be alive, the lighter ones recommend the opposite. The features are sorted based on their importance for this specific mutant; for example, `numAssertInTC` is the most important feature, and `ppmaintainabilityIndexNC` is the least important one. According to the results, `numAssertInTC` is fewer than 14, suggesting that the number of assertions in test classes that check the mutant is not enough. The developer needs to increase the number of corresponding assertions. On the other hand, `numTestCover` is greater than three, indicating that the number of distinct test cases covering the mutant is likely enough. It means that the developer does not need to add new test cases. Instead, she should focus on improving existing test cases. For example, `numExecuted` is fewer than 26, and the model considers that it is quite low. Therefore, EPMT requires her to increase the number of executions. We can also interpret other interesting aspects from Figure 8. The *x*-axis shows the prediction probabilities. For example, the number of assertions in the test class has the prediction probability of 0.18, while the number of executions has the prediction probability of 0.06, suggesting that the number of assertions in the test class is three times more important than the number of executions for that very mutant. Moreover, based on the results, `mmhalsteadDifficulty` is relatively low, indicating that the method on which the mutant is located is not complex. However, `ppmaintainabilityIndexNC` is large, and the model considers that the package is not maintainable. Therefore, it might be a good idea for her to refactor the package.

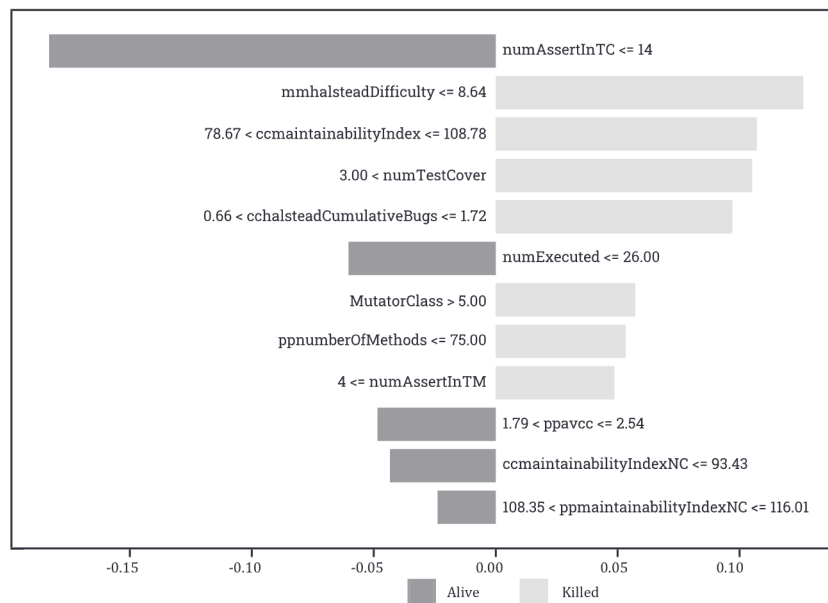


Figure 8. Explanation for a survived mutant.

4.4. Discussion

Here, we discuss the key findings of the paper as well as their implications for both researchers and practitioners. In particular, we recommend several actions such as removing unreached mutants from the data set, providing an explainable machine learning model and focusing on how many times each individual line is executed.

The results provided in this paper indicate that the prior PMT studies performed poorly on reached mutants, highlighting the fact that researchers should remove unreached mutants when reporting their results. Reporting results for the whole data set, including trivial mutants like unreached ones, could be misleading. Furthermore, we found out that using unreached mutants in the learning process has a detrimental effect on the results. Therefore, we suggest that researchers remove unreached mutants even from the train set.

The result interpretation demonstrates that the number of executions is of paramount importance in the prediction process. It suggests whether a mutant execution result (i.e. alive or killed) is affected by how many times the mutant is run. In other words, there is a difference between two mutants M_1 and M_2 with the different number of executions. Literature has shown that mutants are the valid substitution for defects [5,15]. As a result, researchers could also further investigate this feature (i.e. `numExecuted`) on software defect prediction.

EPMT is based on the combination of random forest and gradient boosting, which is a popular technique in industry [107]. Unfortunately, there is a gap between the software industry and research, especially in testing areas. We exploited the combination of different classifications to bridge the gap. We suggest researchers investigate this technique in other software engineering areas (e.g. defect prediction). We conjecture that it would also improve those areas.

For answering RQ3, we provided explanations for one specific mutant. We used LIME to give interpretable explanations, guiding developers to understand why mutants are likely to be killed or alive. In our example (i.e. Figure 8), the oracle aspect of the test suite, `numAssertInTc`, is more important than the number of distinct test cases that cover the mutant. Jiarpakdee *et al.* [97] suggested that model-agnostic techniques (e.g. LIME) should be used to produce easy, meaningful explanations in the area of defect prediction. We use the same argue and recommend researchers and practitioners to exploit such techniques in PMT as the ultimate goal of researchers is to help practitioners develop better software.

Like other machine learning techniques, there are two options when someone faces the results proposed by predictive models: the *optimistic* and *pessimistic* approach. As for the first one, she accepts the results. As for the second one, she accepts the only predictions with high predictability (e.g. 20% of the mutants) and needs to run mutation testing for the remaining mutants [29].

5. THREATS TO VALIDITY

Here, we review threats to the validity of EPMT. We categorize the threats into three categories, namely, internal validity, external validity and dependability [108].

5.1. Internal validity

Our results are based on the data set introduced by Mao *et al.* [39]. Therefore, the chosen projects and the values of independent variables determined by other researchers could affect our results. We assessed EPMT based on evaluation metrics such as AUC, MCC and BA. However, there are a plethora of metrics for evaluating a machine learning model. We mitigated this threat by choosing metrics that have unbiased towards an imbalanced data set [78,109]. We also neglected equivalent mutants in the prediction process. Such mutants certainly would survive because they are equivalent to the original program. However, finding equivalent mutants has been proved to be an undecidable problem [23,25]. We relied on Autorank [100], a Python package for comparison of multiple classifiers, to perform multiple statistical tests. The outcome of these tests could have an impact on our results, yet Autorank follows the well-known guideline provided by Demsar [99].

5.2. External validity

As for external validity, the data set we used is specifically gathered for Java projects. Therefore, it is not applicable to other programming languages. In the evaluation part, we compared our results with the best PMT model, that is, random forest. As a result, there might be another approach performing better than the suggested one. However, we also implemented machine learning models in the previous papers [29,39]. To reduce this threat, we implemented other machine learning models such as logistic regression and deep neural networks. To save space, we avoid reporting their results in this paper, yet they are available at our online appendix [110]. Note that we also compared EPMT to the best machine learning model implemented in the previous research. The data set we used in this paper contains method-level mutants. Therefore, the results are not applicable to class-level mutants. MuJava is a mutation testing tool that is capable of generating class-level mutants [111]. Future research could investigate the impact of such mutants on PMT.

5.3. Dependability

Reproducibility is a key factor in assessing a paper [112-114]. Similar to Mao *et al.* and Zhang *et al.* [29,39], we make all our source code publicly available for other researchers and practitioners to reproduce the results and to further investigate PMT [110]. PMT is a new method, and there are lots of opportunities for other researchers to expand this technique.

6. RELATED WORK

The term unreached mutant comes from the RIPR notion [1]. RIPR is a model showing under which conditions a fault leads to the observed failure. First, a test executes the fault (*reachability*). Second, having executed the faulty statement, the program state is incorrect (*infection*). Third, the program output is incorrect (*propagation*). Fourth, a developer observes at least a portion of the incorrect program state (*revealability*). This term also has been used in previous literature [28,115]. The term uncovered mutant has been defined in the literature with the same meaning as well [41,42,50].

Duque-Torres *et al.* [116] used PMT to investigate whether it is possible to transfer the trained model to a new programming language like C. For this purpose, they replicated the Zhang *et al.* work [29] and then extended PMT to predict mutant execution results on one industrial project written in the C programming language. They concluded that PMT can be transferred successfully. However, the same problem holds for this study. They did not remove unreached mutants when they reported their results.

Random forest has also performed best in other areas related to reducing the cost of mutation testing. For example, Chekam *et al.* [37] proposed an approach to selecting mutants that are fault revealing. A mutant is said to be fault revealing if the mutant is not equivalent to the original program and unveils a program fault. They defined 28 static features of the program and introduced a machine learning approach (i.e. random forest) to predict such mutants. They evaluated their approach on Codeflaws and CoREBench [117,118], which are programs written in the C programming language. They also, for the first time, proposed a concept of mutant prioritization. A mutant with a higher probability of revealing faults has a higher priority for being executed in the mutant prioritization.

Jalbert and Bradbury [62] proposed a machine learning approach for predicting mutation score. They used SVM to predict the mutation score of methods and classes. In particular, they predicted three categories for mutation score: low, medium and high. They exploited static features such as SLOC, cyclomatic complexity (CC) and depth of inheritance tree (DIT). They evaluated their approach on a single project with more than 25,000 lines of code. Our paper is different from Jalbert and Bradbury [62] as we predict each mutant execution result instead of the overall mutation score.

Grano *et al.* [10] used machine learning to estimate mutation score. They extracted 67 software metrics from five categories: test smells, code quality metrics, test quality metrics, code smells and readability. They exploited random forest as a classification algorithm. The model was evaluated on 18 Java projects. When they used both dynamic and static features as input to random forest, they

achieved AUC of 95%. However, a model relying on only static features obtained AUC of 86%. The results show that static features are a good estimator of the mutation score.

Niedermayr and Wagner defined the *minimal stack distance*, the shortest path between a method and a test case in a stack trace, as a new metric for assessing test effectiveness [14]. They used random forest to predict the mutation score of methods. They also exploited eight features such as line coverage and the number of branches as input to the machine learning model. They empirically evaluated their approach on 21 Java projects. The results show that the correlation between minimal stack distance and mutation score in one of the projects is strong, in nine projects are medium and the remaining are weak. The results of the machine learning model are promising. However, there is a subtle difference between our paper and Niedermayr's and Wagner's work [14]. We predict individual mutants instead of estimating the mutation score of methods.

7. CONCLUSIONS

In this paper, we investigated the impact of unreached mutants on the results of previous research regarding PMT. Specifically, we found that such mutants can drastically reduce the performance of state-of-the-art techniques in terms of AUC, MCC and BA. Then we proposed an approach based on the combination of random forest and gradient boosting to handle that issue. The results show that EPMT outperforms the other techniques. Finally, we examined the most important features and found some contradiction when the impact of unreached mutants was considered. For instance, *mmhalsteadDifficulty*, a method-level Halstead difficulty metric, is the second most important feature in EPMT. However, this metric does not even appear on the most important features in the other techniques. The results provided in this paper highlight the need for an interpretable machine learning model. We provided an example (i.e. a survived mutant) and its interpretation using LIME to show why the mutant is predicted to be alive.

For future work, we are going to extend the data set to improve the results further and see the difference in the interpretation of EPMT. We also have a plan to add other metrics related to the abstract syntax tree to enrich the data set and study the impact of such metrics on PMT. Prior research demonstrated that observability metrics correlate with mutation score [119]. We are going to implement those metrics and examine the relationship between them and PMT.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in PUMT at <https://github.com/aaghamohammadi/PUMT>.

REFERENCES

1. Ammann P, Offutt J. *Introduction to Software Testing*. Cambridge University Press, 2016.
2. Jia Y, Harman M. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*. 2011; **37**(5): 649–678.
3. Offutt J. A mutation carol: past, present and future. *Information and Software Technology*. 2011; **53**(10): 1098–1107.
4. Aghamohammadi A, Mirian-Hosseinabadi S-H, Jalali S. Statement frequency coverage: a code coverage criterion for assessing test suite effectiveness. *Information and Software Technology*. 2021; **129**: 106426.
5. Andrews JH, Briand LC, Labiche Y. Is mutation an appropriate tool for testing experiments? In *Proceedings of the 27th International Conference on Software Engineering - ICSE*. ACM Press, 2005; 402–411.
6. Andrews JH, Briand LC, Labiche Y, Namin AS. Using mutation analysis for assessing and comparing testing coverage criteria. *IEEE Transactions on Software Engineering*. 2006; **32**(8): 608–624.
7. Chekam TT, Papadakis M, Traon YL, Harman M. An empirical study on mutation, statement and branch coverage fault revelation that avoids the unreliable clean program assumption. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*. IEEE, 2017; 597–608.
8. Gligoric M, Groce A, Zhang C, Sharma R, Alipour MA, Marinov D. Guidelines for coverage-based comparisons of non-adequate test suites. *ACM Transactions on Software Engineering and Methodology*. 2015; **24**(4): 1–33.
9. Gopinath R, Jensen C, Groce A. Code coverage for suite evaluation by developers. In *Proceedings of the 36th International Conference on Software Engineering - ICSE*. ACM Press, 2014; 72–82.
10. Grano G, Palomba F, Gall HC. Lightweight assessment of test-case effectiveness using source-code-quality indicators. *IEEE Transactions on Software Engineering*. 2019; **47**(4): 1–17.

11. Hemmati H. How effective are code coverage criteria? In *Proceedings of the International Conference on Software Quality, Reliability and Security*. IEEE, 2015; 151–156.
12. Inozemtseva L, Holmes R. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering - ICSE*. ACM Press, 2014; 435–445.
13. Li N, Praphamontrirong U, Offutt J. An experimental comparison of four unit test criteria: mutation, edge-pair, all-uses and prime path coverage. In *Proceedings of the International Conference on Software Testing, Verification, and Validation Workshops - ICSTW*. IEEE, 2009; 220–229.
14. Niedermayr R, Wagner S. Is the stack distance between test case and method correlated with test effectiveness? In *Proceedings of the Evaluation and Assessment on Software Engineering*. ACM Press, 2019; 189–198.
15. Papadakis M, Shin D, Yoo S, Bae D-H. Are mutation scores correlated with real fault detection? In *Proceedings of the 40th International Conference on Software Engineering*. ACM Press, 2018; 537–548.
16. Someoliyi KE, Jalali S, Mahdih M, Mirian-Hosseiniabadi S-H. Program state coverage: a test coverage metric based on executed program states. In *Proceedings of the 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2019; 584–588.
17. Zhang Y, Mesbah A. Assertions are strongly correlated with test suite effectiveness. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering - ESEC/FSE*. ACM Press, 2015; 214–224.
18. Zhu Q, Panichella A, Zaidman A. A systematic literature review of how mutation testing supports quality assurance processes. *Software Testing, Verification and Reliability*. 2018; **28**(6): e1675.
19. Papadakis M, Kintis M, Zhang J, Jia Y, Traon YL, Harman M. Mutation testing advances: an analysis and survey. *Advances in Computers*. 2019; **112**: 275–378.
20. Petrović G, Ivanković M. State of mutation testing at google. In *Proceedings of the 40th International Conference on Software Engineering Software Engineering in Practice - ICSE-SEIP*. ACM Press, 2018; 163–171.
21. Pizzoleto AV, Ferrari FC, Offutt J, Fernandes L, Ribeiro M. A systematic literature review of techniques and metrics to reduce the cost of mutation testing. *Journal of Systems and Software*. 2019; **157**: 110388.
22. Praphamontrirong U, Offutt J. Finding redundancy in web mutation operators. In *Proceedings of the International Conference on Software Testing, Verification and Validation Workshops - ICSTW*. IEEE, 2017; 134–142.
23. Frankl PG, Iakounenko O. Further empirical studies of test effectiveness. In *Proceedings of the 6th ACM SIGSOFT International Symposium on Foundations of Software Engineering - SIGSOFT*. ACM Press, 1998; 153–162.
24. Kaminski G, Praphamontrirong U, Ammann P, Offutt J. A logic mutation approach to selective mutation for programs and queries. *Information and Software Technology*. 2011; **53**(10): 1137–1152.
25. Offutt J, Pan J. Detecting equivalent mutants and the feasible path problem. In *Proceedings of 11th Annual Conference on Computer Assurance*. COMPASS. IEEE, 1996; 224–236.
26. Papadakis M, Henard C, Harman M, Jia Y, Traon YL. Threats to the validity of mutation-based test assessment. In *Proceedings of the 25th International Symposium on Software Testing and Analysis - ISSTA*. ACM Press, 2016; 354–365.
27. Dallilo LD, Pizzoleto AV, Ferrari FC. An evaluation of internal program metrics as predictors of mutation operator score. In *Proceedings of the IV Brazilian Symposium on Systematic and Automated Software Testing - SAST*. ACM Press, 2019; 12–21.
28. Yu M, Ma Y-S. Possibility of cost reduction by mutant clustering according to the clustering scope. *Software Testing, Verification and Reliability*. 2018; **29**(1-2): e1692.
29. Zhang J, Zhang L, Harman M, Hao D, Jia Y, Zhang L. Predictive mutation testing. *IEEE Transactions on Software Engineering*. 2019; **45**(9): 898–918.
30. Devroey X, Perrouin G, Papadakis M, Legay A, Schobbens P-Y, Heymans P. Featured model-based mutation analysis. In *Proceedings of the 38th International Conference on Software Engineering - ICSE*. ACM Press, 2016; 655–666.
31. Gopinath R, Mathis B, Zeller A. If you can't kill a supermutant, you have a problem. In *Proceedings of the International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2018; 18–24.
32. Ma Y-S, Kim S-W. Mutation testing cost reduction by clustering overlapped mutants. *Journal of Systems and Software*. 2016; **115**: 18–30.
33. Fernandes L, Ribeiro M, Carvalho L, Gheyi R, Mongiovi M, Santos A, Cavalcanti A, Ferrari F, Maldonado JC. Avoiding useless mutants. In *Proceedings of the International Conference on Generative Programming: Concepts and Experiences - GPCE*. ACM Press, 2017; 187–198.
34. Iida C, Takada S. Reducing mutants with mutant killable precondition. In *Proceedings of the International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2017; 128–133.
35. Just R, Schweiggert F. Higher accuracy and lower run time: efficient mutation analysis using non-redundant mutation operators. *Software Testing, Verification and Reliability*. 2014; **25**(5-7): 490–507.
36. Steimann F, Thies A. From behaviour preservation to behaviour modification. In *Proceedings of the 32nd International Conference on Software Engineering - ICSE*. ACM Press, 2010; 425–434.
37. Chekam TT, Papadakis M, Bissyandé TF, Traon YL, Sen K. Selecting fault revealing mutants. *Empirical Software Engineering*. 2019; **25**(1): 434–487.
38. Offutt J, Rothermel G, Zapf C. An experimental evaluation of selective mutation. In *Proceedings of the 15th International Conference on Software Engineering - ICSE*. ACM Press, 1993; 100–107.
39. Mao D, Chen L, Zhang L. An extensive study on cross-project predictive mutation testing. In *Proceedings of the Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2019; 160–171.

40. Zhang J, Wang Z, Zhang L, Hao D, Zang L, Cheng S, Zhang L. Predictive mutation testing. In *Proceedings of the 25th International Symposium on Software Testing and Analysis - ISSTA 2016*. ACM Press, 2016; 342–353.
41. Just R, Schweiggert F, Kapfhammer GM. MAJOR: An efficient and extensible tool for mutation analysis in a Java compiler. In *Proceedings of the International Conference on Automated Software Engineering - ASE*. IEEE, 2011; 612–615.
42. Mateo PR, Usaola MP. Reducing mutation costs through uncovered mutants. *Software Testing, Verification and Reliability*. 2014; **25**(5-7): 464–489.
43. Rish I. An empirical study of the naive Bayes classifier. In *Proceedings of the Workshop on Empirical Methods in Artificial Intelligence - IJCAI*, 2001; 41–46.
44. Burges CJC. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*. 1998; **2**(2): 121–167.
45. Rokach L, Maimon O. *Data Mining with Decision Trees—Theory and Applications*, Series in Machine Perception and Artificial Intelligence, vol. 69. World Scientific, 2007.
46. Just R. The major mutation framework: efficient and scalable mutation analysis for java. In *Proceedings of the International Symposium on Software Testing and Analysis - ISSTA*. ACM Press, 2014; 433–436.
47. Just R, Ernst MD, Fraser G. Efficient mutation analysis by propagating and partitioning infected execution states. In *Proceedings of the International Symposium on Software Testing and Analysis - ISSTA*. ACM Press, 2014; 315–326.
48. Just R, Kapfhammer GM, Schweiggert F. Using conditional mutation to increase the efficiency of mutation analysis. In *Proceeding of the 6th International Workshop on Automation of Software Test - AST*. ACM Press, 2011; 50–56.
49. Just R, Kapfhammer GM, Schweiggert F. Do redundant mutants affect the effectiveness and efficiency of mutation analysis? In *Proceedings of the Fifth International Conference on Software Testing, Verification and Validation - ICST*. IEEE, 2012; 720–725.
50. Just R, Kapfhammer GM, Schweiggert F. Using non-redundant mutation operators and test suite prioritization to achieve efficient and scalable mutation analysis. In *Proceedings of the 23rd International Symposium on Software Reliability Engineering - ISSRE*. IEEE, 2012; 11–20.
51. Just R, Kurtz B, Ammann P. Inferring mutant utility from program context. In *Proceedings of the 26th International Symposium on Software Testing and Analysis - ISSTA*. ACM Press, 2017; 284–294.
52. Kintis M, Papadakis M, Papadopoulos A, Valvis E, Malevris N, Traon YL. How effective are mutation testing tools? An empirical analysis of java mutation testing tools with manual analysis and real faults. *Empirical Software Engineering*. 2017; **23**(4): 2426–2463.
53. Laurent T, Ventresque A. PIT-HOM: an extension of pitest for higher order mutation analysis. In *Proceeding of the International Conference on Software Testing, Verification and Validation Workshops - ICSTW*. IEEE, 2019; 83–89.
54. Trautsch F, Herbold S, Grabowski J. Are unit and integration test definitions still valid for modern java projects? An empirical study on open-source projects. *Journal of Systems and Software*. 2020; **159**: 110421.
55. Smith BH, Williams L. On guiding the augmentation of an automated test suite via mutation analysis. *Empirical Software Engineering*. 2008; **14**(3): 341–369.
56. Bowes D, Hall T, Harman M, Jia Y, Sarro F, Wu F. Mutation-aware fault prediction. In *Proceedings of the 25th International Symposium on Software Testing and Analysis - ISSTA*. ACM Press, 2016; 330–341.
57. Herbold S, Trautsch A, Grabowski J. Global vs. local models for cross-project defect prediction. *Empirical Software Engineering*. 2016; **22**(4): 1866–1902.
58. Herbold S, Trautsch A, Grabowski J. A comparative study to benchmark cross-project defect prediction approaches. *IEEE Transactions on Software Engineering*. 2018; **44**(9): 811–833.
59. Tantithamthavorn C, Hassan AE, Matsumoto K. The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Transactions on Software Engineering*. 2018; **46**: 1200–1219.
60. Tantithamthavorn C, McIntosh S, Hassan AE, Matsumoto K. An empirical comparison of model validation techniques for defect prediction models. *IEEE Transactions on Software Engineering*. 2017; **43**(1): 1–18.
61. Tantithamthavorn C, McIntosh S, Hassan AE, Matsumoto K. The impact of automated parameter optimization on defect prediction models. *IEEE Transactions on Software Engineering*. 2019; **45**(7): 683–711.
62. Jalbert K, Bradbury JS. Predicting mutation score using source code and test suite metrics. In *Proceedings of the First International Workshop on Realizing AI Synergies in Software Engineering - RAISE*. IEEE, 2012; 42–46.
63. Pizzoleto AV, Ferrari FC, Dallilo LD, Offutt J. SiMut: exploring program similarity to support the cost reduction of mutation testing. In *Proceedings of the International Conference on Software Testing, Verification and Validation Workshops - ICSTW*. IEEE, 2020; 264–273.
64. da Cruz RC, Eler MM. An empirical analysis of the correlation between CK metrics, test coverage and mutation score. In *Proceedings of the 19th International Conference on Enterprise Information Systems*. SCITEPRESS - Science and Technology Publications, 2017; 341–350.
65. JHawk. Jhawk list of metrics, 2020. <http://www.virtualmachinery.com/Jhawkmetricslist.htm>, Last accessed on 8/13/2020.
66. Geurts P, Ernst D, Wehenkel L. Extremely randomized trees. *Machine Learning*. 2006; **63**(1): 3–42.
67. Iba W, Langley P. Induction of one-level decision trees. In *Proceedings of the Ninth International Workshop on Machine Learning - ML*. Morgan Kaufmann, 1992; 233–240.

68. Shalev-Shwartz S, Ben-David S. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.
69. Quinlan JR. *C4.5: Programs for Machine Learning*. Morgan Kaufmann, 1993.
70. Zhou Z-H, Feng J. Deep forest: towards an alternative to deep neural networks. In *Proceedings of the Twenty-sixth International Joint Conference on Artificial Intelligence - IJCAI*, 2017; 3553–3559.
71. Lee Y. Handwritten digit recognition using K nearest-neighbor, radial-basis function, and backpropagation neural networks. *Neural Computation*. 1991; 3(3): 440–449.
72. Goodfellow IJ, Bengio Y, Courville AC. *Deep Learning*, Adaptive Computation and Machine Learning. MIT Press, 2016.
73. Bengio Y, LeCun Y, Henderson D. Globally trained handwritten word recognizer using spatial representation, convolutional neural networks, and hidden Markov models. In *Proceedings of the Advances in Neural Information Processing Systems*. Morgan Kaufmann, 1993; 937–944.
74. Basili VR. Software modeling and measurement: the goal/question/metric paradigm. *Technical Report*, USA, 1992.
75. Powers DM. Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*. 2011; 2: 37–63.
76. Fawcett T. An introduction to ROC analysis. *Pattern Recognition Letters*. 2006; 27(8): 861–874.
77. Baldi P, Brunak S, Chauvin Y, Andersen CAF, Nielsen H. Assessing the accuracy of prediction algorithms for classification: an overview. *Bioinformatics*. 2000; 16(5): 412–424.
78. Brodersen KH, Ong CS, Stephan KE, Buhmann JM. The balanced accuracy and its posterior distribution. In *Proceedings of the 20th International Conference on Pattern Recognition*. IEEE, 2010; 3121–3124.
79. James G, Witten D, Hastie T, Tibshirani R. *An Introduction to Statistical Learning*. Springer: New York, 2013.
80. Ripley BD. *Pattern Recognition and Neural Networks*. Cambridge University Press, 1996.
81. Russell SJ, Norvig P. *Artificial Intelligence—A Modern Approach, Third International Edition*. Pearson Education, 2010.
82. Haixiang G, Yijing L, Shang J, Mingyun G, Yuanyue H, Bing G. Learning from class-imbalanced data: review of methods and applications. *Expert Systems with Applications*. 2017; 73: 220–239.
83. Khoshgoftaar TM, Gao K, Seliya N. Attribute selection and imbalanced data: problems in software defect prediction. In *Proceedings of the International Conference on Tools with Artificial Intelligence*. IEEE, 2010; 137–144.
84. Malhotra R, Khanna M. An empirical study for software change prediction using imbalanced data. *Empirical Software Engineering*. 2017; 22(6): 2806–2851.
85. Wang S, Yao X. Using class imbalance learning for software defect prediction. *IEEE Transactions on Reliability*. 2013; 62(2): 434–443.
86. He H, Bai Y, Garcia EA, Li S. ADASYN: adaptive synthetic sampling approach for imbalanced learning. In *Proceedings of the International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*. IEEE, 2008; 1322–1328.
87. Breiman L. Random forests. *Machine Learning*. 2001; 45(1): 5–32.
88. Murphy KP. *Machine Learning—A Probabilistic Perspective*, Adaptive Computation and Machine Learning Series. MIT Press, 2012.
89. Mohri M, Rostamizadeh A, Talwalkar A. *Foundations of Machine Learning*. MIT Press, 2018.
90. Altmann A, Tološi L, Sander O, Lengauer T. Permutation importance: a corrected feature importance measure. *Bioinformatics*. 2010; 26(10): 1340–1347.
91. Gregorutti B, Michel B, Saint-Pierre P. Correlation and variable importance in random forests. *Statistics and Computing*. 2016; 27(3): 659–678.
92. Guyon I, Weston J, Barnhill S, Vapnik V. Gene selection for cancer classification using support vector machines. *Machine Learning*. 2002; 46(1/3): 389–422.
93. Zwillinger D, Kokoska S. *CRC Standard Probability and Statistics Tables and Formulae*. CRC Press, 1999.
94. McKinney W. Pandas: a foundational python library for data analysis and statistics. *Python for High Performance and Scientific Computing*. 2011; 14(9): 1–9.
95. Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, VanderPlas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E. Scikit-learn: machine learning in Python, 2012. CoRR, abs/1201.0490.
96. Dam HK, Tran T, Ghose A. Explainable software analytics. In *Proceedings of the 40th International Conference on Software Engineering New Ideas and Emerging Results - ICSE-NIER*. ACM Press, 2018; 53–56.
97. Jiarpakdee J, Tantithamthavorn C, Dam HK, Grundy J. An empirical study of model-agnostic techniques for defect prediction models. *IEEE Transactions on Software Engineering*. 2020; 1–18.
98. Ribeiro MT, Singh S, Guestrin C. “Why should i trust you?” In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM Press, 2016; 1135–1144.
99. Demsar J. Statistical comparisons of classifiers over multiple data sets. *The Journal of Machine Learning Research*. 2006; 7: 1–30.
100. Herbold S. Autorank: a Python package for automated ranking of classifiers. *Journal of Open Source Software*. 2020; 5(48): 2173.
101. Shapiro SS, Wilk MB. An analysis of variance test for normality (complete samples). *Biometrika*. 1965; 52(3/4): 591.
102. Friedman M. A comparison of alternative tests of significance for the problem of m rankings. *The Annals of Mathematical Statistics*. 1940; 11(1): 86–92.

103. Nemenyi P. Distribution-free multiple comparisons. In *Biometrics*, 1962; 263.
104. Cliff N. Dominance statistics: ordinal analyses to answer ordinal questions. *Psychological Bulletin*. 1993; **114**(3): 494.
105. McCabe TJ, Butler CW. Design complexity measurement and testing. *Communications of the ACM*. 1989; **32**(12): 1415–1425.
106. Weyuker EJ. Evaluating software complexity measures. *IEEE Transactions on Software Engineering*. 1988; **14**(9): 1357–1365.
107. Ulyanov D. How to win a data science competition, 2020. <https://www.coursera.org/learn/competitive-data-science>, Last accessed on 11/29/2020.
108. Feldt R, Magazinius A. Validity threats in empirical software engineering research—an initial survey. In *Proceedings of the 22nd International Conference on Software Engineering & Knowledge Engineering*. Knowledge Systems Institute Graduate School, 2010; 374–379.
109. Yao J, Shepperd M. 2020. Assessing software defection prediction performance: why using the Matthews correlation coefficient matters,; 120–129.
110. Aghamohammadi A. Online appendix, 2020. <https://github.com/aaghamohammadi/PUMT>, Last accessed on 8/13/2020.
111. Ma Y-S, Offutt J, Kwon YR. MuJava: an automated class mutation system. *Software Testing, Verification and Reliability*. 2005; **15**(2): 97–133.
112. Herbold S, Trautsch F, Harms P, Herbold V, Grabowski J. Experiences with replicable experiments and replication kits for software engineering research. *Advances in Computers*. 2019; **113**: 315–344.
113. Shepperd M, Ajienka N, Counsell S. The role and value of replication in empirical software engineering results. *Information and Software Technology*. 2018; **99**: 120–132.
114. Trautsch F, Herbold S, Makedonski P, Grabowski J. Addressing problems with replicability and validity of repository mining studies through a smart data platform. *Empirical Software Engineering*. 2017; **23**(2): 1036–1083.
115. Kim S-W, Ma Y-S, Kwon Y-R. Combining weak and strong mutation for a noninterpretive Java mutation system. *Software Testing, Verification and Reliability*. 2012; **23**(8): 647–668.
116. Duque-Torres A, Doliashvili N, Pfahl D, Ramler R. Predicting survived and killed mutants. In *Proceedings of the International Conference on Software Testing, Verification and Validation Workshops - ICSTW*. IEEE, 2020; 274–283.
117. Böhme M, Roychoudhury A. CoREBench: studying complexity of regression errors. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis - ISSTA 2014*. ACM Press, 2014; 105–115.
118. Tan SH, Yi J, Yulis, Mechtaev S, Roychoudhury A. Codeflaws: a programming competition benchmark for evaluating automated program repair tools. In *Proceedings of the International Conference on Software Engineering Companion - ICSE*. IEEE, 2017; 180–182.
119. Zhu Q, Zaidman A, Panichella A. How to kill them all: an exploratory study on the impact of code observability on mutation testing. *Peer J Pre Prints*. 2019; **7**: 1–26.